SpawnHub Ingestion API — Developer Guide

This guide is the starting point for integrating any AI agent framework with SpawnHub. After following it, your agents will appear as animated avatars in the SpawnHub renderer in real time, with full replay support.

Table of Contents

1. How it works
2. Authentication — get your API key
3. Choosing an integration path
4. OTLP path — LangChain & LangGraph
5. OTLP path — AutoGen / AG2
6. OTLP path — CrewAI
7. OTLP path — Google ADK
8. OTLP path — Semantic Kernel
9. Direct events path — OpenAI Agents SDK
10. Direct events path — n8n
11. Direct events path — custom adapter
12. Session grouping and workflow IDs
13. Choosing a pattern
14. Avatar personas
15. Watching the live stream (WebSocket)
16. Replaying past sessions
17. Troubleshooting

---

1. How it works

```
Your agent code
     │
     ├── OTLP spans  (LangChain, AutoGen, CrewAI, Google ADK, Semantic Kernel)
     │        │
     │        ▼
     │   POST /v1/traces
     │
     └── GameEvents  (OpenAI Agents SDK, n8n, custom)
              │
              ▼
         POST /v1/events
              │
              ▼
   SpawnHub ingestion (translates + persists)
              │
       ┌──────┴──────┐
       ▼             ▼
  WebSocket bus   PostgreSQL
       │             │
       ▼             ▼
  Renderer       Replay API
 (live avatars)  (scrub history)
```

Every span or event you send becomes an avatar action in the renderer:

What your agent does	What the renderer shows
Agent is invoked	Avatar appears in the world
LLM call happens	Avatar plays thinking animation
Tool is called	Avatar performs action
Vector store lookup	Avatar walks to the library
Agent finishes	Avatar returns to idle

---

2. Authentication — get your API key

All write endpoints require an `X-SpawnHub-Key` header. Read endpoints (WebSocket, replay) require no authentication.

Local development (Docker stack)

The local stack runs with no auth enforcement by default. You can omit `X-SpawnHub-Key` entirely, or use any placeholder value.

Provisioning a key via Admin API

```
# 1. Create a subscription (company)
curl -X POST http://admin.localhost/admin/api/subscriptions \
  -H "Content-Type: application/json" \
  -d '{"company_name": "Acme Corp", "plan": "free_trial"}'
# → { "subscription_id": "sub-abc123", ... }

# 2. Create a tenant (team / project) — generates an API key
curl -X POST http://admin.localhost/admin/api/tenants \
  -H "Content-Type: application/json" \
  -d '{"subscription_id": "sub-abc123", "tenant_name": "Team Alpha"}'
# → { "tenant_id": "t-xyz456", "api_key": "spwnhub_Xk9...", ... }
```

Use the returned `api_key` value in the `X-SpawnHub-Key` header on all ingestion requests.

---

3. Choosing an integration path

Framework	Path	Effort
LangChain / LangGraph	OTLP → `/v1/traces`	~5 lines of setup code
AutoGen / AG2	OTLP → `/v1/traces`	~5 lines of setup code
CrewAI	OTLP → `/v1/traces`	2 env vars
Google ADK	OTLP → `/v1/traces`	2 env vars

Semantic Kernel (Python)	OTLP → `/v1/traces`	~5 lines of setup code
Semantic Kernel (C#)	OTLP → `/v1/traces`	~10 lines of setup code
OpenAI Agents SDK	Hooks → `/v1/events`	Install adapter package
n8n	HTTP Request node → `/v1/events`	Configure 1 node per agent step
Custom / anything else	POST → `/v1/events`	Write a thin adapter

Rule of thumb: if your framework emits native OTEL, use the OTLP path — it requires less code and automatically captures token counts, tool names, and model names. If it doesn't, POST GameEvents directly to `/v1/events`.

---

4. OTLP path — LangChain & LangGraph

LangChain and LangGraph have native OTEL support since March 2025 via the `opentelemetry-instrumentation-langchain` package.

Install

```
pip install opentelemetry-sdk \
            opentelemetry-exporter-otlp-proto-http \
            opentelemetry-instrumentation-langchain
```

Setup

```python
from opentelemetry import trace
from opentelemetry.sdk.trace import TracerProvider
from opentelemetry.sdk.trace.export import BatchSpanProcessor
from opentelemetry.exporter.otlp.proto.http.trace_exporter import OTLPSpanExporter
from opentelemetry.instrumentation.langchain import LangChainInstrumentor

# Point the OTLP exporter at SpawnHub
provider = TracerProvider()
provider.add_span_processor(
    BatchSpanProcessor(
        OTLPSpanExporter(
            endpoint="http://ingest.localhost/v1/traces",
            headers={"X-SpawnHub-Key": "your-api-key"},
        )
    )
)
trace.set_tracer_provider(provider)

# Instrument LangChain — must be called before any chain/graph is built
LangChainInstrumentor().instrument()
```

Tag a multi-agent run (recommended)

Without `pipeline.run_id`, each agent in a LangGraph graph gets its own session. Add the run ID so all agents appear together in the renderer:

```
import uuid
from opentelemetry import trace as otel_trace

tracer = otel_trace.get_tracer("my-app")

with tracer.start_as_current_span("pipeline-root") as root_span:
    run_id = str(uuid.uuid4())
    root_span.set_attribute("pipeline.run_id", run_id)
    root_span.set_attribute("pipeline.pattern", "orchestrator")

    # Run your LangGraph graph inside this span context —
    # all child spans will inherit the trace context
    result = graph.invoke({"input": "Tell me about AI trends"})
```

LangGraph — StateGraph example

```
from langgraph.graph import StateGraph, END
from typing import TypedDict

class State(TypedDict):
    messages: list

def research_node(state: State) -> State:
    # your LLM call here — LangChain instrumentation captures it automatically
    ...

def write_node(state: State) -> State:
    ...

graph = StateGraph(State)
graph.add_node("research", research_node)
graph.add_node("write", write_node)
graph.add_edge("research", "write")
graph.add_edge("write", END)
graph.set_entry_point("research")

app = graph.compile()

# Wrap in a root span so all nodes share pipeline.run_id
with tracer.start_as_current_span("pipeline-root") as span:
    span.set_attribute("pipeline.run_id", str(uuid.uuid4()))
    span.set_attribute("pipeline.pattern", "sequential")
    app.invoke({"messages": []})
```

---

5. OTLP path — AutoGen / AG2

AutoGen v0.4+ has built-in OTEL support via `OtelTracingConfig`.

Install

```
pip install pyautogen \
            opentelemetry-sdk \
            opentelemetry-exporter-otlp-proto-http
```

Setup

```
from opentelemetry.sdk.trace import TracerProvider
from opentelemetry.sdk.trace.export import BatchSpanProcessor
from opentelemetry.exporter.otlp.proto.http.trace_exporter import OTLPSpanExporter
from autogen import AssistantAgent, UserProxyAgent
from autogen.otel import OtelTracingConfig

# Configure OTLP exporter
provider = TracerProvider()
provider.add_span_processor(
    BatchSpanProcessor(
        OTLPSpanExporter(
            endpoint="http://ingest.localhost/v1/traces",
            headers={"X-SpawnHub-Key": "your-api-key"},
        )
    )
)

# Apply to AutoGen
tracing_config = OtelTracingConfig(tracer_provider=provider)

assistant = AssistantAgent(
    name="AssistantAgent",
    llm_config={"model": "gpt-4o"},
    tracing_config=tracing_config,
)

user_proxy = UserProxyAgent(
    name="UserProxy",
    tracing_config=tracing_config,
)
```

Tag the conversation as a single session

```
from opentelemetry import trace

tracer = trace.get_tracer("my-autogen-app")

with tracer.start_as_current_span("autogen-session") as span:
    session_id = "conv-" + str(uuid.uuid4())
```

```
    span.set_attribute("pipeline.run_id", session_id)
    span.set_attribute("pipeline.pattern", "conversational")

    user_proxy.initiate_chat(assistant, message="Summarise AI research from 2025")
```

AG2 (fork of AutoGen)

AG2 uses the same API. Replace `autogen` imports with `ag2`:

```
from ag2 import AssistantAgent, UserProxyAgent
from ag2.otel import OtelTracingConfig
```

---

6. OTLP path — CrewAI

CrewAI supports OTEL via environment variables — no code changes required.

Install

```
pip install crewai opentelemetry-exporter-otlp-proto-http
```

Setup via environment variables

```
export OTEL_EXPORTER_OTLP_ENDPOINT="http://ingest.localhost/v1/traces"
export OTEL_EXPORTER_OTLP_HEADERS="X-SpawnHub-Key=your-api-key"
export OTEL_SERVICE_NAME="my-crew"
```

Or set them in your `.env` file and load with `python-dotenv`.

Tag the run

CrewAI automatically sets `gen_ai.system = crewai`. Add SpawnHub-specific attributes via a custom callback or by wrapping crew execution in a root span:

```
import uuid
from opentelemetry import trace
from crewai import Crew, Agent, Task

tracer = trace.get_tracer("my-crew-app")

researcher = Agent(
    role="Senior Researcher",
    goal="Uncover groundbreaking AI technologies",
    backstory="You are an AI research expert.",
    verbose=True,
)

research_task = Task(
    description="Investigate the latest advances in AI agents",
```

```
    expected_output="Structured report with key findings",
    agent=researcher,
)

crew = Crew(agents=[researcher], tasks=[research_task])

with tracer.start_as_current_span("crew-run") as span:
    span.set_attribute("pipeline.run_id", str(uuid.uuid4()))
    span.set_attribute("pipeline.pattern", "sequential")
    result = crew.kickoff()
```

---

7. OTLP path — Google ADK

Google ADK has built-in observability that can export OTEL spans.

Install

```
pip install google-adk \
            opentelemetry-sdk \
            opentelemetry-exporter-otlp-proto-http
```

Setup via environment variables

```
export GOOGLE_GENAI_OBSERVABILITY_ENABLED=true
export OTEL_EXPORTER_OTLP_ENDPOINT="http://ingest.localhost/v1/traces"
export OTEL_EXPORTER_OTLP_HEADERS="X-SpawnHub-Key=your-api-key"
```

Setup via code

```
from opentelemetry.sdk.trace import TracerProvider
from opentelemetry.sdk.trace.export import BatchSpanProcessor
from opentelemetry.exporter.otlp.proto.http.trace_exporter import OTLPSpanExporter
from opentelemetry import trace
import google.generativeai as genai

provider = TracerProvider()
provider.add_span_processor(
    BatchSpanProcessor(
        OTLPSpanExporter(
            endpoint="http://ingest.localhost/v1/traces",
            headers={"X-SpawnHub-Key": "your-api-key"},
        )
    )
)
trace.set_tracer_provider(provider)

# ADK picks up the global tracer provider automatically
```

Run your ADK agent

```
from google.adk.agents import Agent
import uuid

tracer = trace.get_tracer("my-adk-app")

agent = Agent(
    name="DataAgent",
    model="gemini-2.0-flash",
    instruction="You are a helpful data analysis agent.",
)

with tracer.start_as_current_span("adk-run") as span:
    span.set_attribute("pipeline.run_id", str(uuid.uuid4()))
    span.set_attribute("pipeline.pattern", "parallel")
    response = agent.run("Analyse the Q1 sales data")
```

---

8. OTLP path — Semantic Kernel

Semantic Kernel (both Python and C#) supports OTEL via its built-in diagnostics.

Python

Install

```
pip install semantic-kernel \
            opentelemetry-sdk \
            opentelemetry-exporter-otlp-proto-http
```

Setup

```
from opentelemetry.sdk.trace import TracerProvider
from opentelemetry.sdk.trace.export import BatchSpanProcessor
from opentelemetry.exporter.otlp.proto.http.trace_exporter import OTLPSpanExporter
from opentelemetry import trace
import semantic_kernel as sk
from semantic_kernel.connectors.ai.open_ai import OpenAIChatCompletion

provider = TracerProvider()
provider.add_span_processor(
    BatchSpanProcessor(
        OTLPSpanExporter(
            endpoint="http://ingest.localhost/v1/traces",
            headers={"X-SpawnHub-Key": "your-api-key"},
        )
    )
)
trace.set_tracer_provider(provider)
```

```
# Enable SK diagnostics (emits gen_ai.* attributes)
import os
os.environ["SEMANTICKERNEL_EXPERIMENTAL_GENAI_ENABLE_OTEL_DIAGNOSTICS"] = "true"

kernel = sk.Kernel()
kernel.add_service(OpenAIChatCompletion(service_id="gpt4o", ai_model_id="gpt-4o"))
```

Run with session tagging

```
import uuid
from opentelemetry import trace

tracer = trace.get_tracer("my-sk-app")

with tracer.start_as_current_span("sk-run") as span:
    span.set_attribute("pipeline.run_id", str(uuid.uuid4()))
    span.set_attribute("pipeline.pattern", "reflection")
    result = await kernel.invoke(my_function, sk.KernelArguments(input="Hello"))
```

C# (.NET)

NuGet packages

```
<PackageReference Include="Microsoft.SemanticKernel" Version="1.*" />
<PackageReference Include="OpenTelemetry.Exporter.OpenTelemetryProtocol"
Version="1.*" />
<PackageReference Include="OpenTelemetry.Extensions.Hosting" Version="1.*" />
```

Setup in Program.cs

```
using OpenTelemetry;
using OpenTelemetry.Trace;
using Microsoft.SemanticKernel;

// Enable SK telemetry
AppContext.SetSwitch("Microsoft.SemanticKernel.Experimental.GenAI.EnableOTelDiagnostics
true);

// Configure OTLP exporter
var tracerProvider = Sdk.CreateTracerProviderBuilder()
    .AddSource("Microsoft.SemanticKernel*")
    .AddOtlpExporter(opt =>
    {
        opt.Endpoint = new Uri("http://ingest.localhost/v1/traces");
        opt.Headers = "X-SpawnHub-Key=your-api-key";
    })
    .Build();

// Build kernel
```

```
var kernel = Kernel.CreateBuilder()
    .AddOpenAIChatCompletion("gpt-4o", apiKey:
Environment.GetEnvironmentVariable("OPENAI_API_KEY"))
    .Build();
```

Run with session tagging

```
using var activity = new ActivitySource("my-sk-app")
    .StartActivity("sk-run");

activity?.SetTag("pipeline.run_id", Guid.NewGuid().ToString());
activity?.SetTag("pipeline.pattern", "orchestrator");

var result = await kernel.InvokeAsync(myFunction, new KernelArguments { ["input"] =
"Hello" });
```

---

9. Direct events path — OpenAI Agents SDK

The OpenAI Agents SDK does not emit native OTEL. Use the `spawnhub-openai-agents` adapter, which hooks into the SDK's built-in lifecycle callbacks and posts GameEvents to `/v1/events`.

Install

```
pip install spawnhub-openai-agents
```

Setup

```
from agents import Agent, Runner
from spawnhub_openai_agents import SpawnHubTracer

# Attach the tracer — this registers the lifecycle hooks
tracer = SpawnHubTracer(
    endpoint="http://ingest.localhost/v1/events",
    api_key="your-api-key",
    workflow_id="my-openai-pipeline",  # optional — groups sessions
    pattern="orchestrator",
)
tracer.attach()
```

Run your agents normally

```
orchestrator = Agent(
    name="Orchestrator",
    instructions="You coordinate a research pipeline.",
    tools=[research_tool, write_tool],
)
```

```
result = Runner.run_sync(orchestrator, "Research AI agent frameworks in 2026")
```

The tracer automatically emits:

- `agent_spawn` when an agent starts
- `agent_think` on each LLM call
- `agent_action` on each tool call
- `agent_complete` when the agent finishes

Customize the persona

```
tracer = SpawnHubTracer(
    endpoint="http://ingest.localhost/v1/events",
    api_key="your-api-key",
    default_persona={
        "framework": "openai",
        "country": "US",
        "gender": "female",
    },
)
```

---

10. Direct events path — n8n

n8n does not emit OTEL natively. Use HTTP Request nodes to call `/v1/events` around each AI Agent step in your workflow.

Node placement pattern

```
[AI Agent node]
       |
       ▼
[HTTP Request — SpawnHub spawn]
       |
       ▼
[... rest of your workflow ...]
       |
       ▼
[HTTP Request — SpawnHub complete]
```

Spawn event node configuration

Setting	Value
Method	POST
URL	`http://ingest.localhost/v1/events`
Authentication	Header Auth
Header name	`X-SpawnHub-Key`

Header value	`your-api-key`
Content-Type	`application/json`

Body (JSON):

```
[
  {
    "event_type": "agent_spawn",
    "session_id": "={{ $workflow.id }}-={{ $execution.id }}",
    "trace_id": "={{ $workflow.id }}-={{ $execution.id }}",
    "span_id": "={{ $node.name }}-spawn",
    "agent_id": "={{ $node.name }}",
    "agent_name": "={{ $node.name }}",
    "parent_agent_id": null,
    "pattern": "sequential",
    "timestamp": "={{ new Date().toISOString() }}",
    "agent_type": "n8n.ai_agent",
    "persona": {
      "framework": "langchain"
    }
  }
]
```

Complete event node configuration

Same settings as above. **Body:**

```
[
  {
    "event_type": "agent_complete",
    "session_id": "={{ $workflow.id }}-={{ $execution.id }}",
    "trace_id": "={{ $workflow.id }}-={{ $execution.id }}",
    "span_id": "={{ $node.name }}-complete",
    "agent_id": "={{ $node.name }}",
    "agent_name": "={{ $node.name }}",
    "pattern": "sequential",
    "timestamp": "={{ new Date().toISOString() }}",
    "success": true
  }
]
```

n8n with multiple AI agents

If your n8n workflow has several AI Agent nodes, give each a unique `agent_id` and set `parent_agent_id` on child agents to build the hierarchy:

```
{
  "agent_id": "={{ $node.name }}-research",
```

```
  "parent_agent_id": "orchestrator-agent"
}
```

n8n Code node helper function

For complex workflows, paste this into a **Code** node before your agent steps to build all required event fields automatically:

```
// In a Code node — returns helpers usable by downstream nodes

const executionId = `${$workflow.id}-${$execution.id}`;

return [{
  json: {
    sessionId: executionId,
    traceId: executionId,
    makeSpawnEvent: (agentName, parentAgentId = null, pattern = "sequential") => ({
      event_type: "agent_spawn",
      session_id: executionId,
      trace_id: executionId,
      span_id: `${agentName}-spawn`,
      agent_id: agentName,
      agent_name: agentName,
      parent_agent_id: parentAgentId,
      pattern,
      timestamp: new Date().toISOString(),
      agent_type: "n8n.ai_agent",
      persona: { framework: "langchain" },
    }),
    makeCompleteEvent: (agentName, success = true) => ({
      event_type: "agent_complete",
      session_id: executionId,
      trace_id: executionId,
      span_id: `${agentName}-complete`,
      agent_id: agentName,
      agent_name: agentName,
      pattern: "sequential",
      timestamp: new Date().toISOString(),
      success,
    }),
  }
}];
```

---

11. Direct events path — custom adapter

If you are building an adapter for any other framework, POST events directly to `/v1/events`.

Minimal Python adapter

```python
import uuid
import httpx
from datetime import datetime, timezone

SPAWNHUB_URL = "http://ingest.localhost/v1/events"
SPAWNHUB_KEY = "your-api-key"

def now() -> str:
    return datetime.now(timezone.utc).isoformat()

def post_events(events: list[dict]) -> None:
    httpx.post(
        SPAWNHUB_URL,
        json=events,
        headers={"X-SpawnHub-Key": SPAWNHUB_KEY},
        timeout=5,
    ).raise_for_status()

# At the start of each agent run
session_id = str(uuid.uuid4())
agent_id = str(uuid.uuid4())

post_events([{
    "event_type": "agent_spawn",
    "session_id": session_id,
    "trace_id": session_id,
    "span_id": agent_id,
    "agent_id": agent_id,
    "agent_name": "MyAgent",
    "parent_agent_id": None,
    "pattern": "orchestrator",
    "timestamp": now(),
    "agent_type": "custom",
    "persona": {"framework": "openai"},
}])

# When the agent calls an LLM
post_events([{
    "event_type": "agent_think",
    "session_id": session_id,
    "trace_id": session_id,
    "span_id": str(uuid.uuid4()),
    "agent_id": agent_id,
    "agent_name": "MyAgent",
    "pattern": "orchestrator",
    "timestamp": now(),
    "model": "gpt-4o",
    "prompt_tokens": 512,
    "completion_tokens": 128,
}])
```

```python
# When the agent uses a tool
post_events([{
    "event_type": "agent_action",
    "session_id": session_id,
    "trace_id": session_id,
    "span_id": str(uuid.uuid4()),
    "agent_id": agent_id,
    "agent_name": "MyAgent",
    "pattern": "orchestrator",
    "timestamp": now(),
    "tool_name": "web_search",
    "tool_input": {"query": "latest AI news"},
}])

# When the agent finishes
post_events([{
    "event_type": "agent_complete",
    "session_id": session_id,
    "trace_id": session_id,
    "span_id": str(uuid.uuid4()),
    "agent_id": agent_id,
    "agent_name": "MyAgent",
    "pattern": "orchestrator",
    "timestamp": now(),
    "success": True,
}])
```

Batching

You can send multiple events in a single POST to reduce HTTP overhead. The order within a batch does not matter — events are sorted by `timestamp` server-side.

```python
post_events([
    spawn_event,
    think_event,
    action_event,
    complete_event,
])
```

---

12. Session grouping and workflow IDs

Sessions

A `session_id` groups all events from **one execution run** into a single replay-able unit in the renderer. All agents, LLM calls, and tool calls in one pipeline run must share the same `session_id`.

Integration path	How to set session_id
OTLP	Set `pipeline.run_id` span attribute. Falls back to `trace_id`.

`/v1/events`	Set `session_id` to the same value on every event in the run.

Use a value that is:

- **Unique per run** — each execution creates a new session in the renderer
- **Stable within a run** — all events in the same run must share it
- **Human-readable** (optional) — helps when browsing past sessions

Good choices: `str(uuid.uuid4())`, conversation ID, job ID, execution ID.

Workflows

A `workflow_id` groups multiple sessions under a named pipeline (e.g. "research-bot", "customer-support"). Set it via the `X-Workflow-ID` header on every request:

```
headers = {
    "X-SpawnHub-Key": "your-api-key",
    "X-Workflow-ID": "research-bot",
}
```

Or set it on the OTLP exporter for all spans from one service:

```
OTLPSpanExporter(
    endpoint="http://ingest.localhost/v1/traces",
    headers={
        "X-SpawnHub-Key": "your-api-key",
        "X-Workflow-ID": "research-bot",
    },
)
```

---

13. Choosing a pattern

The `pattern` field drives which visual theme the renderer auto-selects when the first agent spawns. It also affects how agent positions are laid out.

Pattern	Use when	Default theme
`orchestrator`	One central agent delegates to specialists	Command Center (star layout)
`sequential`	Agents run one after another in a pipeline	Assembly Line (chain layout)
`parallel`	One dispatcher sends tasks to multiple workers simultaneously	War Room (fan layout)
`conversational`	Agents communicate back and forth with no fixed order	Roundtable (circle layout)
`reflection`	An agent generates output then critiques and revises it	Mirror Loop (pair layout)

If no pattern is set, the renderer defaults to `orchestrator`.

The user can override the auto-selected theme manually via the renderer UI.

---

14. Avatar personas

The `persona` object on `agent_spawn` events customises how the avatar looks. All fields are optional.

```
persona = {
    "name":      "Ibrahim",   # Override display name (default: agent_name)
    "gender":    "male",      # "male" | "female" — affects proportions
    "country":   "SA",        # ISO 3166-1 alpha-2 — adds flag badge
    "framework": "langchain", # Drives avatar colour livery
}
```

Framework liveries

framework	Body colour	Accent	Badge
langchain	#1C1C1E	#F5A623	⛓
langgraph	#1A2A44	#4A90D9	◈
openai	#0D0D0D	#10A37F	◎
google_adk	#1A73E8	#34A853	G
autogen	#0078D4	#50E6FF	A
crewai	#1B3A5C	#D4A855	⚓
anthropic	#CC785C	#EDD9D1	◇
semantic_kernel	default	—	—
(unknown / not set)	#555577	#aaaaff	◆

For the OTLP path, `framework` is auto-detected from `gen_ai.system` — you do not need to set it manually. For `/v1/events`, set it explicitly in the `persona` object.

Per-agent persona for multi-agent runs

Each `agent_spawn` event can carry a different persona, so different agents in the same session can have different colours, names, and flag badges.

---

15. Watching the live stream (WebSocket)

The renderer connects to this endpoint automatically. If you are building your own visualisation or monitoring tool, connect directly:

```
const ws = new WebSocket("ws://ingest.localhost/ws");

ws.onmessage = ({ data }) => {
```

```
  const event = JSON.parse(data);
  console.log(event.event_type, event.agent_name, event.session_id);
};

ws.onclose = () => setTimeout(() => reconnect(), 2000); // auto-reconnect
```

Multiple clients can connect simultaneously — each gets an independent copy of every event (fan-out). There is no authentication on the WebSocket.

Event sequence you will see for a typical run

```
agent_spawn   — avatar appears
agent_think   — thinking animation starts
agent_action  — tool use animation
agent_think   — thinking again
agent_complete — returns to idle
```

---

16. Replaying past sessions

All sessions are persisted and available via the REST replay API.

List sessions

```
import httpx

sessions = httpx.get("http://ingest.localhost/sessions").json()
# [
#   {
#     "session_id": "run-xyz-001",
#     "started_at": "2026-04-22T10:00:00Z",
#     "ended_at":   "2026-04-22T10:00:45Z",
#     "event_count": 22,
#     "agent_count": 3,
#     "agent_names": "Orchestrator,ResearchAgent,WriterAgent",
#     "pattern": "orchestrator",
#     "workflow_id": "research-bot"
#   },
#   ...
# ]
```

Load all events for a session

```
events = httpx.get(
    f"http://ingest.localhost/sessions/{session_id}/events"
).json()
# GameEvent[] ordered by timestamp ascending
```

The SpawnHub renderer exposes a replay bar that lets you scrub through any past session at 1×, 2×, or 4× speed. No additional integration is required — as long as your events include accurate `timestamp` values, replay works automatically.

---

17. Troubleshooting

Renderer shows "Disconnected" / no avatars appear

1. Check the ingestion service is running: `make infra-ps` — look for `spawnhub-ingestion`.
2. Verify your exporter endpoint is correct: `http://ingest.localhost/v1/traces` (no trailing slash).
3. Check the ingestion logs: `make infra-logs SERVICE=ingestion` — look for `Accepted X spans`.
4. Confirm the Docker stack is up: `make infra-up`.
5. For OTLP, try setting `OTEL_EXPORTER_OTLP_INSECURE=true` if TLS errors appear.

Spans are ingested but no avatars appear

The translator only produces events from spans it can classify. Check that at least one span has `gen_ai.operation.name = invoke_agent` (or a span name containing "agent").

Run a quick test to see what events are being produced:

```
curl -s "http://ingest.localhost/sessions" | python3 -m json.tool
```

If sessions appear there, then:

```
curl -s "http://ingest.localhost/sessions/<session_id>/events" | python3 -m
json.tool
```

If no `agent_spawn` event appears in the list, your framework is not emitting an `invoke_agent` span. Use the `/v1/events` path instead and manually emit `agent_spawn` at the start of each agent invocation.

All agents appear in separate sessions (not grouped)

Set `pipeline.run_id` to the same value on every span in the run. Without it, each span's `trace_id` is used as the session — and frameworks often create a new trace per agent invocation.

```
root_span.set_attribute("pipeline.run_id", "my-run-001")
```

Avatar has wrong colour / no framework badge

Check `gen_ai.system` is being set on your spans. For CrewAI and Google ADK, this is set automatically. For LangChain, it requires `opentelemetry-instrumentation-langchain`. For `/v1/events`, set `persona.framework` explicitly.

n8n: events not arriving / 401 errors

- Verify the HTTP Request node has `X-SpawnHub-Key` set in **Headers**, not in **Query parameters**.
- In n8n expressions, use `={{ }}` syntax for dynamic values.
- Check the n8n execution log for the HTTP Request node's response body.

OTLP errors with `application/x-protobuf`

Switch the exporter to JSON:

```
from opentelemetry.exporter.otlp.proto.http.trace_exporter import OTLPSpanExporter
# This uses JSON by default — not protobuf
exporter = OTLPSpanExporter(endpoint="http://ingest.localhost/v1/traces")
```

Events arriving but replay scrubber is empty

Replay events load only after selecting a session from the dropdown in the renderer. If the session list is empty, check the ingestion service has write access to PostgreSQL: `make infra-logs SERVICE=ingestion` — look for database errors on startup.